

Embedded C Cheat Sheet

Prüfungsrelevante Syntax, Speicher & Hardware

1. Build-Prozess & C-Toolchain

Werkzeug	Eingabeformat	Ausgabeformat
C-Präprozessor	C-Code (.c, .h)	Erweiterter C-Code (Makros aufgelöst)
C-Compiler	Erweiterter C-Code	Assemblercode (.s oder .asm)
Assembler	Assemblercode	Maschinencode / Object-Code (.o)
Linker	Maschinencode (.o + Libs)	Ausführbare Datei (.elf, .bin, .exe)

2. Speicher, Variablen & Pointer

Speicherbereiche (Memory Layout)

- **Stack:** Für lokale Variablen, Funktionsaufrufe, Parameter. Wird automatisch verwaltet.
- **Heap:** Für dynamischen Speicher (via malloc(), calloc()). Muss manuell mit free() freigegeben werden (sonst: **Memory Leak**).

Pointer & Call-by-Reference

Pointer speichern Speicheradressen. Wichtig für die Rückgabe mehrerer Werte aus einer Funktion (Call-by-Reference).

- &variable: "Adresse-von" Operator (gibt den Pointer zurück).
- *pointer: Dereferenzierungs-Operator (greift auf den Wert an der Adresse zu).

```
// Funktion mit Call-by-Reference (Ergebnis über Pointer zurückgeben)
```

```
int div_int(int a, int b, int *res) {  
    if (b == 0) return 1; // Fehlercode  
    *res = a / b;          // Wert in Speicher schreiben  
    return 0;             // SUCCESS  
}
```

```
// Aufruf in main()
```

```
int result;  
if (div_int(10, 2, &result) == 0) {  
    // result ist jetzt 5  
}
```

Zuweisungen bei Pointern (Klausurfalle!)

```
struct data *old = malloc(sizeof(struct data));  
struct data *new = malloc(sizeof(struct data));
```

```
old = new; // FALSCH! Pointer wird umgebogen. Speicher von 'old' ist verloren  
(Leak), 'new' wird später doppelt befreit!
```

```
*old = *new; // RICHTIG! Kopiert den tatsächlichen INHALT der Struct von 'new' nach  
'old'.
```

3. Structs, Alignment & Padding

Alignment (Speicherausrichtung)

- **Definition:** Variablen werden an Adressen abgelegt, die ein Vielfaches ihrer Größe sind (32-Bit/4-Byte-Variablen an Adressen wie 0x...00, 0x...04, 0x...08).

- **Padding:** Der Compiler fügt unsichtbare “Füllbytes” in Structs ein, um das Alignment der nachfolgenden Variablen sicherzustellen.

```
typedef struct {
    uint8_t temp;    // 1 Byte
    // --- 3 Bytes Padding ---
    uint32_t press;  // 4 Bytes (Muss auf 4-Byte-Grenze liegen!)
    uint8_t humid;   // 1 Byte
    // --- 3 Bytes Padding am Ende (für Array-Alignment) ---
} sensor_data_t;
// sizeof(sensor_data_t) = 12 Bytes! (Nicht 6)
```

KLAUSURFALLE: Structs mit memcmp() vergleichen

memcmp(a, b, sizeof(struct)) vergleicht den Speicher Byte für Byte. Da die Padding-Bytes uninitialisiert sind und zufälligen “Müll” enthalten können, schlägt memcmp oft fehl, selbst wenn die eigentlichen Daten (temp, press, humid) identisch sind! **Lösung:** Immer Element für Element manuell mit if (a->temp == b->temp) vergleichen.

4. Hardwarenahe Programmierung (Sehr wichtig!)

Direkter Register-Zugriff

Um Hardware-Register zu beschreiben, muss eine feste Hex-Adresse in einen Pointer gecastet und dereferenziert werden. **Zwingend nötig:** Das Schlüsselwort volatile. Es verbietet dem Compiler, Lese-/Schreibzugriffe wegzuroptimieren.

```
// Schreibe den 16-Bit-Wert 0xFFFF an die Adresse 0x00F3B404
*((volatile uint16_t *)0x00F3B404) = 0xFFFF;

// Lese einen 32-Bit-Wert von einer Adresse
uint32_t val = *((volatile uint32_t *)0x40001000);
```

Bitmanipulation (Bitmasking)

Standard-Aufgabe in Embedded C: Einzelne Bits in einem Register verändern, **ohne** die anderen anzufassen. Sei N die Bit-Position (0 = LSB).

- **Bit Setzen (1):** REG |= (1 << N);
- **Bit Löschen (0):** REG &= ~(1 << N);
- **Bit Umschalten (Toggle):** REG ^= (1 << N);
- **Bit Prüfen:** if (REG & (1 << N)) { ... }

5. Nebenläufigkeit, Synchronisation & Interrupts

Semaphoren & Mutexe

Werden genutzt, um Ressourcen (z.B. Variablen, Hardware) vor gleichzeitigem Zugriff mehrerer Tasks/Threads zu schützen.

- **Mutex (Mutual Exclusion):** Erfordert 1 binäre Semaphore. (Sperrt den kritischen Bereich).
- **Producer-Consumer-Modell:** Erfordert mindestens 2 (meist 3) Semaphores:
 1. Zählsemaphore für freie Plätze (Free Space).
 2. Zählsemaphore für verfügbare Elemente (Items).
 3. Mutex (Schutz des Puffers/Arrays).

Interrupt Service Routinen (ISR)

- Eine ISR unterbricht das Hauptprogramm bei Hardware-Ereignissen (z.B. Timer, Button).

- **Wichtige Regeln für ISRs:**

1. So kurz und schnell wie möglich ausführen.
2. Keine blockierenden Aufrufe (kein printf, malloc, delay).
3. Globale Variablen, die in der ISR geändert und in der main() gelesen werden, MÜSSEN als volatile deklariert sein!

6. Wichtige Standard-Funktionen (I/O)

printf()

Zur formatierten Textausgabe (benötigt `#include <stdio.h>`). **Embedded-Hinweis:** printf verbraucht oft sehr viel Speicher (Flash) und Laufzeit. Auf Mikrocontrollern muss die Ausgabe meist hardwarenah auf eine serielle Schnittstelle (z.B. UART) umgeleitet werden (**Retargeting**).

Wichtige Format-Specifier:

- %d / %i: Vorzeichenbehaftete Ganzzahl (int)
- %u: Vorzeichenlose Ganzzahl (unsigned int)
- %x / %X: Hexadezimal (klein-/großgeschrieben) -> **Sehr wichtig für Register!**
- %f: Fließkommazahl (float/double)
- %c: Einzelnes Zeichen (char)
- %s: Zeichenkette (String, char-Pointer)
- %p: Speicheradresse (Pointer)

Formatierung & Padding (Klausur-Klassiker):

- %04x: Hex-Wert mit führenden Nullen auf exakt 4 Stellen auffüllen.
- %8d: Integer rechtsbündig auf eine Breite von 8 Zeichen setzen (mit Leerzeichen aufgefüllt).
- %.2f: Float auf 2 Nachkommastellen runden.

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint16_t reg_val = 255;
    float temp = 23.456;
    int *ptr = (int*)0x20000000;

    // Ausgabe: "Register: 0x00FF | Temp: 23.46"
    printf("Register: 0x%04X | Temp: %.2f\n", reg_val, temp);

    // Ausgabe von Speicheradressen
    printf("Adresse liegt bei: %p\n", (void*)ptr);

    return 0;
}
```